

OOP Basics

Object-Oriented Programming, or OOP, is a programming style where software is modeled as a collection of objects. Each object combines data and behavior. OOP is useful because real-world systems are often easier to understand as interacting entities: a `Student`, `BankAccount`, `Car`, `User`, `Order`, or `Payment`.

OOP is especially important for placements because interviewers often test whether you understand the ideas behind code, not just syntax. You should be able to explain what a class is, why encapsulation matters, how inheritance differs from polymorphism, and when abstraction is useful.

1. Class

A class is a blueprint, template, or design for creating objects. It defines what data an object will store and what operations the object can perform.

Example:

```
class Student {  
    String name;  
    int rollNumber;  
  
    void study() {  
        System.out.println(name + " is studying");  
    }  
}
```

Here, `Student` is a class. It says that every student object can have a `name`, a `rollNumber`, and a behavior called `study`.

Key Points

- A class does not represent one actual entity by itself.
- It defines the structure and behavior of objects.
- A class can contain fields, methods, constructors, nested classes, and access modifiers.
- In many languages, classes support encapsulation, inheritance, and polymorphism.

Real-Life Analogy

A class is like a building plan. The plan is not the building itself, but many buildings can be created from the same plan.

Interview Answer

A class is a user-defined data type that acts as a blueprint for creating objects. It groups data members and member functions into a single unit.

2. Object

An object is an instance of a class. It is a real entity created from the class blueprint.

Example:

```
Student s1 = new Student();
s1.name = "Aman";
s1.rollNumber = 101;
s1.study();
```

Here, `s1` is an object of the `Student` class.

Key Points

- Objects occupy memory.
- Each object has its own state.
- Objects interact with one another through methods.
- An object has identity, state, and behavior.

Identity, State, and Behavior

Identity means the object is uniquely distinguishable from other objects.

State means the values stored inside the object.

Behavior means the actions the object can perform.

Example:

```
class Car {
    String color;
    int speed;

    void accelerate() {
        speed += 10;
    }
}
```

For a `Car` object:

- Identity: the specific car object in memory
- State: `color`, `speed`
- Behavior: `accelerate`

Interview Answer

An object is a runtime instance of a class. It contains actual values for the fields defined in the class and can use the methods defined by the class.

3. Encapsulation

Encapsulation means wrapping data and methods together inside a single unit, usually a class, and restricting direct access to the internal data.

The goal is to protect object data from unauthorized or invalid access.

Example:

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Here, `balance` is private. It cannot be changed directly from outside the class. The class controls how money is deposited.

Why Encapsulation Is Important

- It protects data from invalid modification.
- It improves security.
- It hides internal implementation details.
- It makes code easier to maintain.
- It allows validation before changing data.
- It reduces dependency between different parts of the program.

Bad Example Without Encapsulation

```
class BankAccount {  
    public double balance;  
}  
  
BankAccount account = new BankAccount();  
account.balance = -5000;
```

This is dangerous because anyone can assign an invalid balance.

Better Example With Encapsulation

```
class BankAccount {  
    private double balance;  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        }  
    }  
}
```

Now the object protects its own rules.

Encapsulation vs Data Hiding

Encapsulation is the broader idea of bundling data and methods together.

Data hiding is the technique of restricting direct access to data, often using private variables.

Interview Answer

Encapsulation is the process of binding data and methods together in a class and restricting direct access to the data using access modifiers. It helps protect object state and maintain data integrity.

4. Abstraction

Abstraction means showing only essential details and hiding unnecessary internal implementation.

It focuses on what an object does, not how it does it.

Example:

When you drive a car, you use the steering wheel, accelerator, and brakes. You do not need to know the internal working of the engine to drive it. That is abstraction.

Code Example

```
abstract class Shape {  
    abstract double area();  
}  
  
class Circle extends Shape {  
    double radius;  
  
    Circle(double radius) {  
        this.radius = radius;  
    }  
  
    double area() {  
        return 3.14 * radius * radius;  
    }  
}
```

```
}  
}
```

The **Shape** class defines that every shape must have an **area**, but it does not specify how the area is calculated.

Abstraction Can Be Achieved Using

- Abstract classes
- Interfaces
- Encapsulated methods
- Well-designed APIs

Why Abstraction Is Important

- It reduces complexity.
- It hides implementation details.
- It makes systems easier to use.
- It helps achieve loose coupling.
- It allows implementation changes without affecting users of the class.

Abstraction vs Encapsulation

Encapsulation hides data.

Abstraction hides implementation details.

Encapsulation is about how data is protected inside a class.

Abstraction is about exposing only the necessary features to the outside world.

Interview Answer

Abstraction is the OOP concept of hiding internal implementation details and exposing only essential functionality. It helps reduce complexity and improves code maintainability.

5. Inheritance

Inheritance is the mechanism by which one class acquires the properties and behaviors of another class.

The existing class is called the parent class, base class, or superclass.

The new class is called the child class, derived class, or subclass.

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

Here, `Dog` inherits the `eat` method from `Animal`.

Why Inheritance Is Used

- To reuse code.
- To represent an `is-a` relationship.
- To support method overriding.
- To enable runtime polymorphism.
- To organize related classes.

Is-A Relationship

Inheritance should be used when the child class is a specialized version of the parent class.

Examples:

- Dog is an Animal.
- Car is a Vehicle.
- Manager is an Employee.

Bad example:

- Car is an Engine.

A car has an engine, so composition is better here.

Types of Inheritance

Different languages support different forms of inheritance.

Single Inheritance

One child class inherits from one parent class.

```
class Dog extends Animal {  
}
```

Multilevel Inheritance

A class inherits from a child class, forming a chain.

```
class Animal {  
}  
  
class Mammal extends Animal {  
}  
  
class Dog extends Mammal {  
}
```

Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

```
class Animal {  
}  
  
class Dog extends Animal {  
}  
  
class Cat extends Animal {  
}
```

Multiple Inheritance

A class inherits from multiple parent classes.

C++ supports multiple inheritance with classes.

Java does not support multiple inheritance with classes, but it supports it through interfaces.

```
class A {  
};  
  
class B {  
};  
  
class C : public A, public B {  
};
```

Diamond Problem

The diamond problem occurs in multiple inheritance when a class inherits from two classes that both inherit from the same base class.

Example:



If both **B** and **C** inherit from **A**, and **D** inherits from both **B** and **C**, then **D** may receive two copies of **A** members. This can create ambiguity.

C++ solves this using virtual inheritance.

Java avoids this problem with classes by not allowing multiple class inheritance.

Inheritance vs Composition

Inheritance means **is-a**.

Composition means **has-a**.

Example:

```
class Engine {  
}  
  
class Car {  
    private Engine engine;  
}
```

A car has an engine, so composition is better.

Important Interview Point

Do not use inheritance only for code reuse. Use inheritance when there is a true **is-a** relationship. For many designs, composition is more flexible than inheritance.

Interview Answer

Inheritance is an OOP feature that allows one class to acquire the fields and methods of another class. It promotes code reuse and supports polymorphism when used with method overriding.

6. Polymorphism

Polymorphism means many forms.

In OOP, polymorphism allows the same method, operator, or interface to behave differently depending on the object or input.

Types of Polymorphism

There are two main types:

- Compile-time polymorphism
- Runtime polymorphism

6.1 Compile-Time Polymorphism

Compile-time polymorphism is resolved during compilation.

It is commonly achieved using method overloading or operator overloading.

Method Overloading

Method overloading means having multiple methods with the same name but different parameter lists.

Example:

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

The method name is the same, but the parameters are different.

Rules for Method Overloading

Methods can be overloaded by changing:

- Number of parameters
- Type of parameters
- Order of parameters

Methods cannot be overloaded only by changing the return type.

Invalid example:

```
int add(int a, int b) {  
    return a + b;  
}  
  
double add(int a, int b) {
```

```
    return a + b;  
}
```

This is invalid in Java because the parameter list is the same.

Operator Overloading

Operator overloading means giving operators special behavior for user-defined types.

C++ supports operator overloading.

Java does not support custom operator overloading, except built-in behavior like `+` for strings.

Example in C++:

```
class Complex {  
public:  
    int real, imag;  
  
    Complex operator+(Complex other) {  
        Complex result;  
        result.real = real + other.real;  
        result.imag = imag + other.imag;  
        return result;  
    }  
};
```

6.2 Runtime Polymorphism

Runtime polymorphism is resolved during program execution.

It is commonly achieved using method overriding.

Method Overriding

Method overriding means a child class provides its own implementation of a method already defined in the parent class.

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {
```

```
        System.out.println("Dog barks");
    }
}
```

Now, when the method is called on a **Dog** object, the child class method runs.

Runtime Polymorphism Example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound();
        a2.sound();
    }
}
```

Output:

```
Bark
Meow
```

Although the reference type is **Animal**, the actual object type decides which method runs.

Overloading vs Overriding

Feature	Overloading	Overriding
Polymorphism type	Compile-time	Runtime

Feature	Overloading	Overriding
Same class or inheritance	Usually same class	Requires inheritance
Method name	Same	Same
Parameters	Must be different	Must be same
Return type	Can differ only if parameters differ	Must be same or covariant
Resolution	Compile time	Runtime

Static Binding vs Dynamic Binding

Static binding means the method call is resolved at compile time.

Dynamic binding means the method call is resolved at runtime based on the actual object.

Overloading uses static binding.

Overriding uses dynamic binding.

Interview Answer

Polymorphism is the ability of the same interface or method to behave differently depending on the object or input. Compile-time polymorphism is achieved through overloading, while runtime polymorphism is achieved through overriding.

7. Constructors

A constructor is a special method that is automatically called when an object is created.

Its main purpose is to initialize the object.

Example:

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Object creation:

```
Student s = new Student("Riya", 21);
```

Properties of Constructors

- A constructor has the same name as the class in Java and C++.
- A constructor does not have a return type.
- A constructor is called automatically when an object is created.
- Constructors can be overloaded.
- Constructors are mainly used to initialize fields.
- If no constructor is defined, many languages provide a default constructor.

Types of Constructors

Default Constructor

A constructor with no parameters.

```
class Student {  
    Student() {  
        System.out.println("Student created");  
    }  
}
```

Parameterized Constructor

A constructor that accepts arguments.

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

Copy Constructor

A constructor that creates a new object by copying another object.

C++ commonly uses copy constructors.

Java does not provide a built-in copy constructor, but you can create one manually.

```
class Student {  
    String name;  
  
    Student(Student other) {  
        this.name = other.name;  
    }  
}
```

```
}  
}
```

Constructor Overloading

Constructor overloading means having multiple constructors with different parameter lists.

```
class Student {  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Use of `this`

The `this` keyword refers to the current object.

It is often used when local variables and instance variables have the same name.

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

Constructor Chaining

Constructor chaining means calling one constructor from another constructor.

```
class Student {  
    String name;  
    int age;
```

```
Student() {  
    this("Unknown", 0);  
}  
  
Student(String name, int age) {  
    this.name = name;  
    this.age = age;  
}  
}
```

Interview Answer

A constructor is a special method that is automatically invoked when an object is created. It is used to initialize the object and can be overloaded to support different ways of creating objects.

8. Destructors

A destructor is a special method used to clean up resources before an object is destroyed.

Destructors are especially important in languages like C++ where memory and resource management are more manual.

Example in C++:

```
class FileHandler {  
public:  
    FileHandler() {  
        cout << "File opened" << endl;  
    }  
  
    ~FileHandler() {  
        cout << "File closed" << endl;  
    }  
};
```

The destructor has the same name as the class but starts with `~`.

Properties of Destructors in C++

- A destructor has the same name as the class with a `~` prefix.
- It has no return type.
- It takes no parameters.
- It cannot be overloaded.
- It is called automatically when an object goes out of scope or is deleted.
- It is used to release resources like memory, file handles, sockets, or database connections.

Why Destructors Are Needed

Objects may acquire resources during their lifetime. If those resources are not released, the program may suffer from memory leaks or resource leaks.

Examples of resources:

- Dynamically allocated memory
- Open files
- Network connections
- Locks
- Database connections

Destructor Example With Dynamic Memory

```
class Array {  
private:  
    int* data;  
  
public:  
    Array(int size) {  
        data = new int[size];  
    }  
  
    ~Array() {  
        delete[] data;  
    }  
};
```

Destructors in Java

Java does not have destructors like C++.

Java has garbage collection, which automatically frees unused memory. However, garbage collection does not automatically close non-memory resources like files or database connections.

For resource cleanup in Java, use:

- `try-with-resources`
- `close()` methods
- `AutoCloseable`

Example:

```
try (FileReader reader = new FileReader("data.txt")) {  
    // use file  
}
```

Important Interview Point

Do not say Java has destructors. Java has garbage collection, but deterministic cleanup is usually done using `try-with-resources` or explicit close methods.

Interview Answer

A destructor is a special method used to release resources when an object is destroyed. C++ supports destructors directly, while Java does not have destructors and relies on garbage collection plus explicit resource management techniques.

9. Access Modifiers

Access modifiers control the visibility and accessibility of classes, fields, methods, and constructors.

They are used to implement encapsulation and protect data.

Common Access Modifiers

Public

Accessible from anywhere.

```
public int age;
```

Use `public` for methods or members that are part of the external API.

Private

Accessible only within the same class.

```
private double balance;
```

Use `private` for internal data and implementation details.

Protected

Accessible within the same package and by subclasses.

```
protected String name;
```

Use `protected` carefully. It exposes details to child classes and can make code harder to maintain.

Default or Package-Private in Java

If no access modifier is used in Java, the member is accessible only within the same package.

```
class Student {  
    int marks;  
}
```

Here, `marks` has package-private access.

Java Access Modifier Table

Modifier	Same Class	Same Package	Subclass Different Package	Other Package
<code>private</code>	Yes	No	No	No
default	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

C++ Access Modifiers

C++ mainly uses:

- `public`
- `private`
- `protected`

In C++ classes, members are `private` by default.

In C++ structs, members are `public` by default.

```
class Student {  
private:  
    int marks;  
  
public:  
    void setMarks(int m) {  
        marks = m;  
    }  
};
```

Best Practices

- Keep fields private.
- Expose behavior through public methods.
- Avoid making everything public.
- Use protected only when subclasses truly need access.
- Prefer methods over direct field access.
- Keep class internals hidden unless there is a strong reason to expose them.

Interview Answer

Access modifiers define the visibility of class members. They help implement encapsulation by controlling which parts of the program can access or modify data.

10. The Four Pillars of OOP

The four main pillars of OOP are:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Encapsulation

Protects data by binding it with methods and restricting access.

Abstraction

Hides internal implementation and shows only essential features.

Inheritance

Allows one class to reuse and extend another class.

Polymorphism

Allows the same operation to behave differently for different objects.

11. Important OOP Relationships

Association

Association means one class is connected to another class.

Example:

```
class Teacher {  
}  
  
class Student {  
    Teacher teacher;  
}
```

A student is associated with a teacher.

Aggregation

Aggregation is a weak **has-a** relationship. The child object can exist independently of the parent object.

Example:

```
class Department {  
    List<Teacher> teachers;  
}
```

Teachers can exist even if the department is deleted.

Composition

Composition is a strong **has-a** relationship. The child object depends on the parent object.

Example:

```
class House {  
    private Room room;  
  
    House() {  
        room = new Room();  
    }  
}
```

If the house is destroyed, its rooms are also considered destroyed.

Association vs Aggregation vs Composition

Relationship	Meaning	Dependency
Association	Uses or knows about	Weak
Aggregation	Has-a, but independent	Medium
Composition	Has-a, dependent lifetime	Strong

12. Abstract Class vs Interface

This is a very common placement question.

Abstract Class

An abstract class can have abstract methods and concrete methods.

```
abstract class Animal {  
    abstract void sound();  
  
    void sleep() {  
        System.out.println("Sleeping");  
    }  
}
```

```
    }  
}
```

Interface

An interface defines a contract that classes can implement.

```
interface Flyable {  
    void fly();  
}  
  
class Bird implements Flyable {  
    public void fly() {  
        System.out.println("Bird flies");  
    }  
}
```

Abstract Class vs Interface Table

Feature	Abstract Class	Interface
Purpose	Shared base behavior	Contract or capability
Methods	Abstract and concrete	Abstract, default, static methods depending on language/version
Fields	Can have instance variables	Usually constants in Java
Constructor	Can have constructor	Cannot have constructor in Java
Inheritance	A class can extend one abstract class in Java	A class can implement multiple interfaces
Use when	Classes are closely related	Unrelated classes need common behavior

Simple Rule

Use an abstract class when classes share common state or behavior.

Use an interface when you want to define a capability or contract.

13. Common Interview Questions

What is OOP?

OOP is a programming paradigm based on objects. Objects combine data and behavior. OOP improves modularity, reusability, maintainability, and real-world modeling.

What are the four pillars of OOP?

The four pillars are encapsulation, abstraction, inheritance, and polymorphism.

Difference between class and object?

A class is a blueprint. An object is an instance of that blueprint.

Difference between encapsulation and abstraction?

Encapsulation hides and protects data inside a class.

Abstraction hides implementation details and exposes only necessary functionality.

Difference between overloading and overriding?

Overloading means same method name with different parameters. It is compile-time polymorphism.

Overriding means a child class provides a new implementation of a parent class method. It is runtime polymorphism.

Can constructors be overloaded?

Yes. Constructors can be overloaded by providing different parameter lists.

Can constructors be inherited?

No. Constructors are not inherited, but a child class constructor can call a parent class constructor.

Can a constructor be private?

Yes. Private constructors are used in patterns like Singleton, utility classes, and factory-controlled object creation.

Example:

```
class Singleton {
    private static Singleton instance;

    private Singleton() {
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Can static methods be overridden?

In Java, static methods cannot be overridden. They can be hidden. Method overriding is based on runtime objects, but static methods belong to the class.

Can private methods be overridden?

No. Private methods are not visible to child classes, so they cannot be overridden.

Can final methods be overridden?

No. A final method cannot be overridden.

Can abstract methods be private?

Usually no, because abstract methods must be implemented by subclasses. If a method is private, subclasses cannot access it.

What is dynamic method dispatch?

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at runtime based on the actual object type.

What is coupling?

Coupling refers to the degree of dependency between classes or modules.

Low coupling is preferred because changes in one class are less likely to break other classes.

What is cohesion?

Cohesion refers to how closely related the responsibilities of a class are.

High cohesion is preferred because each class should have a clear and focused responsibility.

What is object slicing?

Object slicing is a C++ concept. It happens when a derived class object is assigned to a base class object by value, causing the derived-specific part to be sliced off.

```
class Animal {  
};  
  
class Dog : public Animal {  
public:  
    int barkVolume;  
};  
  
Dog d;  
Animal a = d;
```

Here, the `Dog` part beyond `Animal` is sliced.

What is a virtual function?

In C++, a virtual function is a function in the base class that can be overridden in a derived class and called polymorphically through a base class pointer or reference.

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal sound";
    }
};
```

Why should destructors be virtual in base classes?

In C++, if a class is intended to be used polymorphically, its destructor should be virtual. Otherwise, deleting a derived object through a base class pointer may not call the derived destructor, causing resource leaks.

```
class Base {
public:
    virtual ~Base() {
    }
};
```

14. Common Mistakes To Avoid

- Saying a class occupies memory exactly like an object. A class is a blueprint; objects occupy memory for instance data.
- Saying Java has destructors. Java has garbage collection, not C++-style destructors.
- Confusing overloading and overriding.
- Saying inheritance is always good. Composition is often better.
- Making all fields public.
- Using inheritance for **has-a** relationships.
- Thinking abstraction and encapsulation are the same.
- Forgetting that constructors do not have a return type.
- Saying private methods are overridden.
- Saying static methods are overridden in Java.

15. Quick Revision Sheet

Concept	Meaning	Example
Class	Blueprint for objects	<code>class Student</code>
Object	Instance of a class	<code>new Student()</code>
Encapsulation	Data protection	private fields with public methods
Abstraction	Hide implementation	interface or abstract class
Inheritance	Reuse and specialize parent class	<code>Dog extends Animal</code>

Concept	Meaning	Example
Polymorphism	Many forms	overridden <code>sound()</code> method
Constructor	Initializes object	<code>Student()</code>
Destructor	Cleans resources	<code>~Student()</code> in C++
Access Modifier	Controls visibility	<code>private</code> , <code>public</code>

16. One-Minute Placement Explanation

Object-Oriented Programming is a programming paradigm based on classes and objects. A class is a blueprint, and an object is an instance of that class. OOP has four major pillars: encapsulation, abstraction, inheritance, and polymorphism. Encapsulation protects data by keeping fields private and exposing controlled methods. Abstraction hides implementation details and exposes only essential behavior. Inheritance allows one class to acquire properties and behavior from another class, representing an `is-a` relationship. Polymorphism allows the same method or interface to behave differently based on the object or input. Constructors initialize objects, destructors clean up resources in languages like C++, and access modifiers control visibility.

17. Best Way To Answer OOP Questions

When answering in interviews:

- Start with a clear definition.
- Give a small real-life analogy.
- Give a short code example if asked.
- Mention why the concept is useful.
- Mention one common mistake or edge case if relevant.

Example answer format:

```
Encapsulation means binding data and methods together and restricting direct access to data. For example, a BankAccount class keeps balance private and exposes deposit or withdraw methods. This protects the object from invalid changes and keeps the internal implementation hidden.
```

18. Final Memory Hooks

- Class: blueprint.
- Object: real instance.
- Encapsulation: protect data.
- Abstraction: hide details.
- Inheritance: reuse through `is-a`.
- Polymorphism: same name, different behavior.
- Constructor: object setup.
- Destructor: object cleanup.
- Access modifier: visibility control.